

LISTA DE EXERCÍCIOS — UNIDADE II

Estrutura e Responsabilidade

Apresentação

A presente lista contempla **trinta exercícios** distribuídos entre os cinco subtópicos da seção *Estrutura e Responsabilidade* da Unidade II. Os enunciados foram elaborados de modo a estimular a transição do raciocínio procedural para o pensamento orientado a objetos, com ênfase na **modelagem de domínio**, na **delimitação de fronteiras de responsabilidade** e na **disciplina de encapsulamento**.

Premissas pedagógicas

- **Linguagem:** TypeScript em modo estrito (`strict: true`), com identificadores e comentários em inglês.
- **Ferramentas:** Node.js, Yarn, Vitest (ou Jest), ESLint e Prettier.
- **Restrições:** todas as soluções devem ser desenvolvidas com **autoria própria**, vedada a utilização de ferramentas de Inteligência Artificial em qualquer etapa de implementação, refatoração ou raciocínio. A apropriação conceitual exige a manipulação direta do código pelo discente.
- **Escopo:** os exercícios são autocontidos no recorte *Estrutura e Responsabilidade*.

Estrutura da lista

Bloco	Subtópico	Exercícios
A	Atributos e métodos	01–06
B	Construtores e inicialização de objetos	07–12
C	Modificadores de acesso	13–18
D	Responsabilidade de classes	19–24
E	Coesão e acoplamento	25–30

Bloco A — Atributos e métodos

Exercício 01 — Carnaubal Monitorado

Contexto. Uma cooperativa de extrativismo do litoral piauiense deseja registrar dados das áreas de carnaubais sob sua supervisão.

Requisitos.

- Modele a classe `CarnaubaGrove` com atributos para identificação da área, município, número estimado de pés, idade média da plantação e quilogramas de cera produzidos no último ciclo.
- Implemente métodos para *(i)* registrar uma nova colheita, *(ii)* calcular a produtividade média por pé e *(iii)* informar se a área é considerada produtiva (limiar definido pelo discente, com justificativa).

Critério de aceitação. A produtividade deve refletir corretamente as colheitas registradas; áreas sem colheitas devem retornar zero, jamais um erro de divisão.

Exercício 02 — Catálogo de Carrancas do Velho Chico

Contexto. Um museu de Parnaíba pretende digitalizar seu acervo de carrancas — esculturas folclóricas tradicionalmente colocadas nas proas das embarcações que navegavam pelo São Francisco.

Requisitos.

- Modele a classe `Figurehead` com atributos para nome da peça, escultor, ano de produção, material predominante (madeira, ferro, gesso) e estado de conservação (categórico: excelente, bom, regular, frágil).
- Implemente métodos que retornem a idade da peça em anos completos (com base na data atual), uma descrição textual padronizada e uma indicação se a peça requer restauro (com base no estado de conservação).

Critério de aceitação. A descrição textual deve ser determinística e adequada para inserção em fichas catalográficas.

Exercício 03 — Cronômetro de Defesa Acadêmica

Contexto. As bancas de TCC da UESPI exigem que cada candidato apresente seu trabalho dentro de um intervalo rigoroso de tempo. Modele o relógio que controla esse processo.

Requisitos.

- Crie a classe `DefenseTimer` com atributos para o tempo total permitido (em segundos), o tempo decorrido e um estado interno (`idle` , `running` , `paused` , `finished`).

- Implemente métodos `start`, `pause`, `resume`, `tick(seconds)` (que avança o tempo decorrido) e `remainingTime`.
- O cronômetro deve **rejeitar** transições inválidas — por exemplo, retomar um cronômetro que nunca foi iniciado.

Critério de aceitação. O método `remainingTime` jamais deve retornar valor negativo; ao atingir o limite, o estado deve transicionar automaticamente para `finished`.

Exercício 04 — Diário de Bordo de Pesquisa de Campo

Contexto. Uma pesquisadora do Delta do Parnaíba registra, diariamente, observações sobre a fauna local. O diário é manipulado por método, jamais por acesso direto.

Requisitos.

- Modele a classe `FieldJournal` com atributos para o nome do pesquisador, a data de início da expedição e uma coleção interna de entradas (cada entrada com data, local e observação textual).
- Implemente métodos para adicionar uma entrada, listar todas as entradas, filtrar por intervalo de datas e contar o total de observações.

Critério de aceitação. A coleção de entradas não deve ser exposta diretamente ao mundo externo; alterações só ocorrem via métodos.

Exercício 05 — Estação Meteorológica de Bolso

Contexto. Um sensor portátil registra temperaturas a cada hora e mantém um histórico das últimas dez leituras.

Requisitos.

- Modele a classe `PocketWeatherStation` com atributos para a leitura atual, a unidade de medida (Celsius ou Fahrenheit) e o histórico circular de leituras.
- Implemente métodos para registrar uma nova leitura, calcular a média das leituras armazenadas, retornar a leitura mais alta e a mais baixa, e converter o histórico para uma representação textual.

Critério de aceitação. Quando o histórico atinge dez leituras, a leitura mais antiga deve ser substituída pela mais recente, sem crescimento ilimitado da estrutura interna.

Exercício 06 — Caderneta de Lembretes Priorizados

Contexto. Um servidor público mantém uma caderneta digital de lembretes administrativos com diferentes níveis de prioridade.

Requisitos.

- Modele a classe `Reminder` (com identificador, texto, prioridade e data de criação) e a classe `ReminderBook` (que agrega lembretes).
- Em `ReminderBook`, implemente métodos para adicionar lembrete, remover por identificador, listar ordenado por prioridade decrescente e contar lembretes pendentes.

Critério de aceitação. O identificador de cada lembrete deve ser gerado internamente; o consumidor da classe não fornece identificadores manualmente.

Bloco B — Construtores e inicialização de objetos

Exercício 07 — Matrícula Universitária Padronizada

Contexto. A pró-reitoria acadêmica solicita que toda matrícula de novo ingressante siga o padrão `AAAA#####`, em que `AAAA` é o ano de ingresso e os seis dígitos finais correspondem a um sequencial.

Requisitos.

- Modele a classe `Enrollment` cujo construtor recebe nome do estudante, código do curso e ano de ingresso, e que **constrói internamente** o número de matrícula.
- O sequencial deve ser controlado por um contador estático interno à classe.
- Reje-se construção com nome vazio, código de curso inexistente em uma lista predefinida ou ano fora do intervalo plausível.

Critério de aceitação. Duas matrículas criadas no mesmo ano não devem possuir o mesmo número.

Exercício 08 — Receita Médica Eletrônica

Contexto. Um sistema clínico recebe receitas eletrônicas com um conjunto mínimo de informações obrigatórias por força de regulamentação.

Requisitos.

- Modele a classe `Prescription` cujo construtor exige paciente, prescritor (nome e CRM), uma lista **não vazia** de medicamentos (cada qual com nome, dosagem e posologia) e a data de emissão.
- A construção deve falhar de modo explícito caso qualquer campo obrigatório esteja ausente ou inválido.

Critério de aceitação. Não deve ser possível instanciar uma receita parcialmente preenchida e completá-la depois.

Exercício 09 — Passaporte Diplomático Imutável

Contexto. Um passaporte diplomático, uma vez emitido, jamais tem seus dados alterados — apenas substituídos por um novo documento.

Requisitos.

- Modele a classe `DiplomaticPassport` em que **todos** os atributos sejam definidos exclusivamente pelo construtor e marcados como `readonly`.
- A classe deve oferecer apenas métodos de consulta, jamais de mutação.
- Implemente um método que retorne se o passaporte está vencido em relação a uma data fornecida.

Critério de aceitação. Tentativas de modificação após a construção devem ser rejeitadas pelo compilador, não em tempo de execução.

Exercício 10 — Embarcação Pesqueira do Litoral

Contexto. Uma colônia de pescadores cadastra suas embarcações com características fixas (capacidade máxima, calado mínimo) e variáveis (carga atual).

Requisitos.

- Modele a classe `FishingBoat` cujo construtor recebe nome da embarcação, capacidade máxima em quilogramas e calado mínimo em metros — atributos imutáveis.
- A carga atual inicia em zero e é alterada por métodos `loadCargo(weight)` e `unloadCargo(weight)`.
- O calado efetivo é **calculado** a partir do calado mínimo e da carga atual, segundo uma fórmula linear estipulada pelo discente.

Critério de aceitação. Carregamentos que excedam a capacidade máxima devem ser rejeitados sem alterar o estado interno.

Exercício 11 — Bilhete de Embarque com Inicialização Derivada

Contexto. Quando um passageiro adquire um bilhete, o portão de embarque é determinado pelo código do voo, segundo uma regra interna da companhia.

Requisitos.

- Modele a classe `BoardingPass` cujo construtor recebe o passageiro e o código do voo, e que **infere** o portão de embarque por meio de um método auxiliar privado.
- O construtor deve também atribuir um número sequencial de embarque, controlado internamente.

Critério de aceitação. A regra de derivação do portão deve ser determinística e testável de modo isolado, sem depender da instanciação completa.

Exercício 12 — Conta Poupança com Construtores Nomeados

Contexto. Uma cooperativa de crédito oferece duas modalidades de abertura de conta poupança: a padrão (saldo inicial zero) e a promocional (com bônus de boas-vindas em datas específicas).

Requisitos.

- Modele a classe `SavingsAccount` cujo construtor primário seja **privado**, expondo dois métodos estáticos de construção: `openStandard(holderName)` e `openWithWelcomeBonus(holderName, bonusAmount)`.
- O propósito é demonstrar o padrão de **construtores nomeados** (*factory methods*), uma vez que TypeScript não permite sobrecarga real de construtores.

Critério de aceitação. Tentativas de construir a conta diretamente, sem passar pelos métodos estáticos, devem ser impedidas pelo modelo.

Bloco C — Modificadores de acesso

Exercício 13 — Cofre Comunitário de Cooperativa

Contexto. Uma cooperativa rural mantém um cofre digital cujo saldo somente pode ser consultado e movimentado pelas operações autorizadas.

Requisitos.

- Modele a classe `CommunalVault` com saldo `private` e histórico de movimentações `private`.
- Exponha apenas os métodos `deposit`, `withdraw`, `getBalance` e `getStatement` (que retorna **uma cópia defensiva** do histórico).
- Saques superiores ao saldo devem ser rejeitados.

Critério de aceitação. Modificações na lista retornada por `getStatement` não devem afetar o estado interno do cofre.

Exercício 14 — Diário Pessoal Cifrado

Contexto. Um diário digital armazena seus registros em forma cifrada e somente revela seu conteúdo após autenticação.

Requisitos.

- Modele a classe `EncryptedDiary` com conteúdo `private`, senha `private` e métodos `lock`, `unlock(password)` e `read()`.
- O método `read` somente retorna conteúdo enquanto o diário estiver desbloqueado; após `lock`, novas leituras exigem nova autenticação.
- A senha **jamais** deve ser exposta em getters, *toString* ou serialização.

Critério de aceitação. Implemente uma rotina de cifragem trivial (por exemplo, deslocamento de caracteres) — o foco do exercício é o encapsulamento, não a criptografia robusta.

Exercício 15 — Urna Eletrônica Simulada

Contexto. Modele uma urna eletrônica didática que apura votos sem jamais expor o voto individual de qualquer eleitor.

Requisitos.

- A classe `BallotBox` deve armazenar internamente uma coleção `private` de votos, cada qual associado a um identificador anônimo de cédula.
- Métodos públicos: `castVote(candidateId)`, `closeBallot()` e `getResults()` (que retorna apenas totais agregados por candidato).
- Após `closeBallot`, novos votos devem ser rejeitados.

Critério de aceitação. Não deve existir qualquer método público que retorne a lista bruta de votos.

Exercício 16 — Carteira Digital de Vacinação

Contexto. A vigilância sanitária mantém a carteira de vacinação do cidadão. Doses só podem ser registradas por unidades de saúde autorizadas.

Requisitos.

- Modele a classe `VaccinationCard` com lista de doses `private`. Cada dose contém vacina, lote, data e unidade aplicadora.
- Métodos públicos: `registerDose`, `getDosesFor(vaccineName)` (cópia defensiva) e `isProtocolComplete(vaccineName, requiredDoses)`.
- Não deve ser possível remover uma dose registrada — o histórico é imutável após inserção.

Critério de aceitação. A imutabilidade do histórico deve ser garantida estruturalmente, não por convenção.

Exercício 17 — Sensor com Calibração Interna

Contexto. Um sensor industrial possui um *offset* de calibração ajustável apenas por procedimento técnico autorizado, não pelo consumidor da classe.

Requisitos.

- Modele a classe `IndustrialSensor` com leitura bruta `public`, leitura calibrada `public` (calculada) e *offset* `protected`.
- Exponha um método `calibrate(referenceValue)` que ajusta o *offset* internamente, sem permitir a definição arbitrária de seu valor por terceiros.
- Justifique no comentário da classe a escolha pelo modificador `protected` em detrimento de `private`, antecipando o uso futuro em hierarquias.

Critério de aceitação. O *offset* não deve ser legível ou escrito por código externo à classe.

Exercício 18 — Cofre de Senhas com Múltiplos Modificadores

Contexto. Um gerenciador de senhas pessoal armazena credenciais e expõe apenas operações controladas.

Requisitos.

- Modele a classe `PasswordVault` que utilize, em conjunto:
 - `private` para a coleção de credenciais e para a senha mestra;
 - `readonly` para o nome do proprietário, definido apenas no construtor;
 - um **método auxiliar** `private` que produza o *hash* de uma senha antes de armazená-la (utilizando uma função de transformação simples definida pelo discente);
 - métodos `public` para `addCredential`, `removeCredential` e `verifyMaster(password)`.

Critério de aceitação. A senha mestra não pode ser obtida por qualquer caminho público; sua verificação ocorre exclusivamente pelo método `verifyMaster`.

Bloco D — Responsabilidade de classes

Observação metodológica para o Bloco D. Os exercícios apresentam, propositalmente, classes que concentram responsabilidades excessivas. A tarefa do discente é (i) identificar as responsabilidades distintas, (ii) propor uma redistribuição em classes coesas e (iii) justificar tecnicamente as decisões. A entrega deve conter, além do código, um diagrama textual ou visual indicando as classes resultantes e suas responsabilidades.

Exercício 19 — A Nota Fiscal Inchada

Contexto. O sistema legado de uma cooperativa contém uma classe `Invoice` com dezenas de métodos: cálculo de impostos, geração de PDF, envio por e-mail, persistência em arquivo, validação de CPF, conversão para JSON, registro em *log* e cálculo de desconto progressivo.

Requisitos.

- Identifique **pelo menos quatro** responsabilidades distintas presentes na classe original.
- Refatore em um conjunto de classes coesas, mantendo `Invoice` como o núcleo de dados e cálculo financeiro essencial.
- A regra é estrita: **nenhuma** das novas classes deve depender de detalhes de implementação das demais — apenas de seus contratos públicos.

Critério de aceitação. A classe `Invoice` resultante deve ser instanciável e calculável sem dependência de impressão, persistência ou rede.

Exercício 20 — Folha de Pagamento Confusa

Contexto. O sistema de RH possui uma classe `Payroll` que calcula salários líquidos, gera contracheques em texto, monta o arquivo de remessa para o banco e produz relatórios gerenciais consolidados.

Requisitos.

- Separe a classe original em **no mínimo três** classes especializadas, cada uma com responsabilidade clara e nomenclatura precisa.
- Demonstre como o cálculo do salário líquido pode ser testado sem que sejam executadas operações de E/S.

Critério de aceitação. A entrega deve incluir uma tabela explicativa relacionando cada responsabilidade da classe original à classe resultante.

Exercício 21 — O Concierge Que Faz Tudo

Contexto. Em um sistema de hotelaria, a classe `Hotel` realiza reservas, registra hóspedes, gerencia disponibilidade de quartos, calcula diárias, processa cobranças, envia confirmações e mantém um histórico de ocupação.

Requisitos.

- Identifique pelo menos **quatro** responsabilidades e proponha uma decomposição em classes (sugestões: `Reservation`, `Guest`, `Room`, `Billing`, `OccupancyLog`).
- A classe `Hotel`, na versão refatorada, deve atuar essencialmente como **orquestradora**, delegando às demais.

Critério de aceitação. A operação completa de uma reserva ainda deve poder ser executada por meio de uma única chamada à `Hotel`, sem exposição interna das classes auxiliares para o consumidor externo.

Exercício 22 — Feirante Polivalente

Contexto. No sistema de uma feira livre municipal, a classe `Vendor` atende clientes, calcula preços, controla estoque, registra entregas e emite cupons fiscais.

Requisitos.

- Realize a separação em pelo menos três classes coesas, justificando técnica e textualmente cada decisão.
- Apresente, na entrega, uma análise das **fronteiras de responsabilidade** entre o feirante (ator do domínio) e os artefatos por ele manipulados (estoque, venda, entrega).

Critério de aceitação. O conceito de “feirante” deve permanecer no domínio modelado; sua responsabilidade, contudo, deve ser limitada à interação humana com o cliente.

Exercício 23 — Sistema de Provas da Disciplina

Contexto. Um sistema acadêmico possui a classe `Exam`, que armazena enunciados, respostas corretas, gabaritos preenchidos pelos discentes, calcula notas, produz estatísticas (média, mediana, desvio-padrão) e exporta os resultados em CSV.

Requisitos.

- Refatore em classes especializadas (`ExamSheet`, `AnswerKey`, `StudentSubmission`, `ExamStatistics`, `CsvExporter` ou denominações equivalentes propostas pelo discente).
- Demonstre que o cálculo estatístico é independente da exportação.

Critério de aceitação. A inclusão futura de um novo formato de exportação (por exemplo, JSON) deve exigir alteração em **uma única** classe.

Exercício 24 — Acervo de Biblioteca Universitária

Contexto. No sistema da biblioteca, a classe `LibraryItem` mistura dados bibliográficos (título, autor, ISBN, edição), informações de acervo físico (localização, exemplares disponíveis) e operações de empréstimo (reservas, devoluções, multas).

Requisitos.

- Distinga conceitualmente o **título bibliográfico** (entidade abstrata) do **exemplar físico** (entidade concreta) e modele cada um em sua classe específica.
- Mova as operações de empréstimo para uma classe dedicada.

Critério de aceitação. Deve ser possível adicionar um novo exemplar sem duplicar dados bibliográficos, e modificar metadados bibliográficos sem percorrer cada exemplar.

Bloco E — Coesão e acoplamento

Exercício 25 — Calculadora de Tarifas com Tabela Trocável

Contexto. Uma empresa de transporte interestadual calcula o valor das passagens de acordo com uma tabela de tarifas que varia por convênio, época do ano ou rota.

Requisitos.

- Modele a classe `FareCalculator` que **não** instancia internamente sua tabela de tarifas, mas a recebe via construtor (injeção de dependência).
- A tabela é representada por uma classe `FareTable` com método `getRate(origin, destination)`.
- Demonstre, em testes, que substituir a tabela altera o resultado do cálculo sem qualquer modificação na `FareCalculator`.

Critério de aceitação. O grau de acoplamento entre `FareCalculator` e `FareTable` deve ser apenas o do contrato público, jamais de implementação.

Exercício 26 — Termostato e Aquecedor Desacoplados

Contexto. Um termostato inteligente decide quando ligar ou desligar o aquecedor com base na temperatura ambiente. Hoje, ele instancia diretamente a classe `ElectricHeater`, o que impossibilita testar seu comportamento sem o aquecedor real.

Requisitos.

- Refatore para que `Thermostat` receba qualquer objeto que satisfaça um pequeno **contrato** (nesta etapa, expresso como `type` literal de função ou classe coesa, sem aprofundamento em `interface`): métodos `turnOn()` e `turnOff()`.
- Disponibilize duas implementações concretas distintas: `ElectricHeater` e `GasHeater`.

Critério de aceitação. Os testes do termostato devem ser executáveis com um aquecedor falso (*fake*), elaborado pelo próprio discente, sem necessidade de mocks de framework.

Exercício 27 — Disparador de Avisos Reorganizado

Contexto. A classe `NotificationDispatcher` instancia diretamente, em seu construtor, uma `EmailSender` — o que impede o uso de outros canais (SMS, *push*, log local) e dificulta os testes.

Requisitos.

- Identifique e nomeie o problema de acoplamento presente.
- Refatore para que o canal de envio seja recebido por construtor.
- Implemente, como demonstração, dois canais alternativos.

Critério de aceitação. A entrega deve incluir um parágrafo explicando, com vocabulário técnico, o que é **alta coesão** e **baixo acoplamento** em relação à versão original e à versão refatorada.

Exercício 28 — Carrinho de Compras e Cupom Fiscal

Contexto. Em um sistema comercial, o **carrinho** apresenta os itens em escolha pelo cliente; o **cupom fiscal** é emitido ao final da compra. Ambas as classes precisam cooperar, mas devem permanecer independentes em sua evolução.

Requisitos.

- Modele `ShoppingCart` e `Receipt` de modo que ambas se comuniquem **somente** por meio de uma estrutura comum mínima (por exemplo, uma lista de itens com nome, quantidade e preço unitário).
- Nenhuma classe deve conter referência a campos privados da outra.

Critério de aceitação. Alterar a regra de cálculo do total no `ShoppingCart` não deve impactar o `Receipt`, e vice-versa.

Exercício 29 — Diagnóstico de Coesão na Classe Estudante

Contexto. Um sistema acadêmico apresenta a classe `Student` com os seguintes atributos: matrícula, nome, curso, período atual, histórico escolar, frequência semanal de exercícios físicos, número de seguidores em redes sociais, IMC, marca preferida de tênis e meta de horas de estudo semanal.

Requisitos.

- Analise criticamente quais atributos comprometem a coesão da classe.
- Reorganize em **uma ou mais** classes separadas, justificando o critério de agrupamento adotado (por exemplo: dimensão acadêmica, dimensão pessoal, dimensão de saúde).
- Apresente, no relatório de entrega, a regra geral utilizada para julgar a aderência de um atributo à classe `Student`.

Critério de aceitação. A classe `Student` resultante deve representar de forma fiel o conceito acadêmico de estudante, sem servir de repositório indiscriminado de dados pessoais.

Exercício 30 — Auditoria de Acoplamento Bidirecional

Contexto. Um sistema de pedidos contém três classes interconectadas: `Order` mantém referência ao `Customer`, `Customer` mantém uma lista de `Order`, e ambos manipulam diretamente atributos de `DeliveryAddress`. Essa configuração produz acoplamento bidirecional excessivo e fragiliza a manutenção.

Requisitos.

- Identifique todos os pontos de **acoplamento bidirecional** e **acoplamento de dados** entre as classes.
- Proponha uma reorganização em que:
 - `Order` mantenha apenas um identificador de cliente, não a referência completa;
 - `Customer` não conheça seus pedidos diretamente — uma classe `OrderRegistry` (ou denominação equivalente) atue como intermediária;
 - `DeliveryAddress` exponha apenas métodos de consulta, jamais permitindo que outras classes alterem seus atributos diretamente.
- Documente, em parágrafo único, as **vantagens** e os **eventuais custos** da redução do acoplamento — pois há sempre contrapartidas, e o discente deve aprender a reconhecê-las.

Critério de aceitação. A entrega deve evidenciar, por meio de testes, que cada classe pode ser instanciada e exercitada de modo autônomo, sem instanciar obrigatoriamente as demais.

Critérios gerais de avaliação

Em consonância com a Seção 7 do Plano de Curso, cada exercício será avaliado segundo:

1. **Correção funcional** — a solução resolve o problema proposto e satisfaz o critério de aceitação.
2. **Qualidade da modelagem orientada a objetos** — adequação dos atributos, métodos e responsabilidades às fronteiras conceituais do domínio.
3. **Disciplina de encapsulamento** — uso pertinente dos modificadores de acesso, com justificativa explícita quando relevante.
4. **Coesão e acoplamento** — evidência de que cada classe responde por uma única dimensão do problema, e que as dependências entre classes são mínimas e explícitas.
5. **Legibilidade e organização** — nomenclatura significativa, métodos curtos, ausência de duplicação evidente.
6. **Coertura de testes** — testes automatizados que verifiquem o comportamento esperado e os casos-limite, com particular atenção às decisões justificadas no enunciado.
7. **Capacidade de justificar decisões técnicas** — em entrevista oral ou parágrafo escrito, conforme estipulado pelo professor.

Orientação final

Os exercícios foram propositalmente formulados de modo a admitir **mais de uma solução defensável**. Espera-se do discente, portanto, não apenas a entrega do código funcional, mas a capacidade de **defender suas escolhas** à luz dos conceitos estudados. A modelagem orientada a objetos é, em última análise, uma atividade de discernimento técnico — e o desenvolvimento desse discernimento é o propósito central desta lista.

Lista elaborada para uso didático na disciplina de Programação Orientada a Objetos do Curso de Tecnologia em Sistemas de Computação da Universidade Estadual do Piauí — UESPI, Campus Parnaíba.